

Introduction to Machine Learning and Data Analysis

Bias-variance trade-off

Mahdi Shafiee Kamalabad and Maarten Cruyff

Content of today's lecture

1. Supervised learning
2. Statistical models
3. Bias vs variance
4. Cross validation
5. The caret package

What are inputs (X) and outputs (Y)?

Input: X

- The information we use to make a prediction, represented by variables.
- Also called features, predictors, or independent variables.

For example: A patient's symptoms, pixel values in an image, a chess board position, square footage of a house

Output: Y

- Y is also called the **target, label, dependent variable, or response**.
- X and Y are connected to the inputs.
- Sometimes we want to understand how Y changes when the inputs change
- We use the X and Y to find patterns, and later predict the output for new inputs.
- We know the correct Y, so it acts like a teacher and helps **supervise** the learning by finding patterns.
- Examples of Y: a medical diagnosis, the identity of a person in a face image, the price of a house.

Supervised Learning

The goal is to learn a rule, or a **model**, that maps inputs (X) to outputs (Y).

$$X \longrightarrow Y$$

- Shows the model many examples where both X and Y are known. This is called **training**.
- The model learns the relationship between X and Y.
- Gives the model a new, unseen input X.
- The model predicts the corresponding output Y.
- Prioritizes predictive performance; interpretability may be optional

Statistical Inference vs. Supervised Learning

Statistical Inference

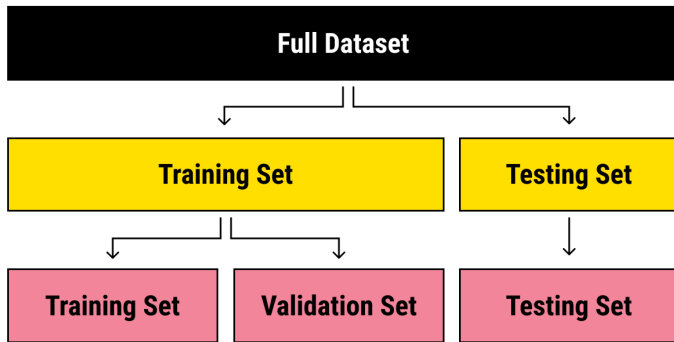
- Tests hypotheses
- Evaluates evidence for or against a theory
- Asks questions such as:
Does intervention X have a statistically significant effect on outcome Y?
- Focuses on understanding relationships and estimating effects
- Emphasizes interpretability and uncertainty, such as:
 - Confidence intervals
 - Standard errors
 - p-values

Training the Model

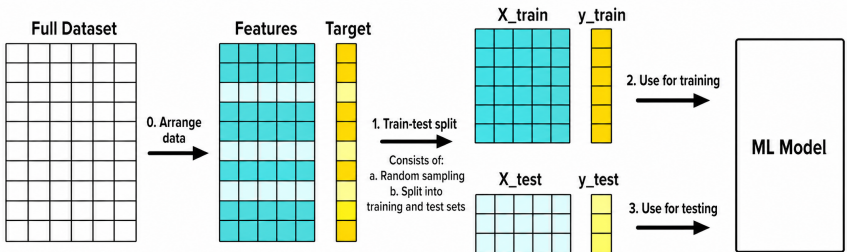
- We split the data into two datasets: a **training dataset** and a **test dataset**
- We show the model many labeled examples, (X, Y) , from the training dataset
- The model learns patterns that connect X to Y
- During training, it improves by reducing its prediction errors
- The trained model is then used to make predictions on the test dataset

Training data and test data

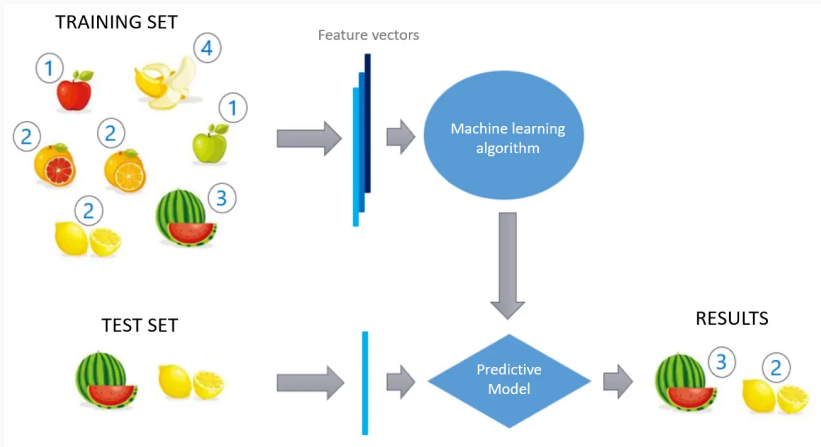
- **Training data:** used to train (fit) the model
- **Test data:** used to evaluate how well the model performs on new data based on the **prediction**.
- Training and test data must be separate
- Good performance on test data means the model can generalize well.



Training data and test data



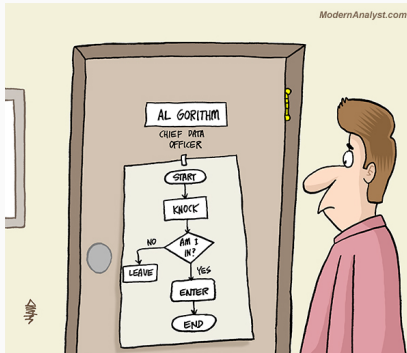
Prediction of an outcome



Algorithm, Model and Rule

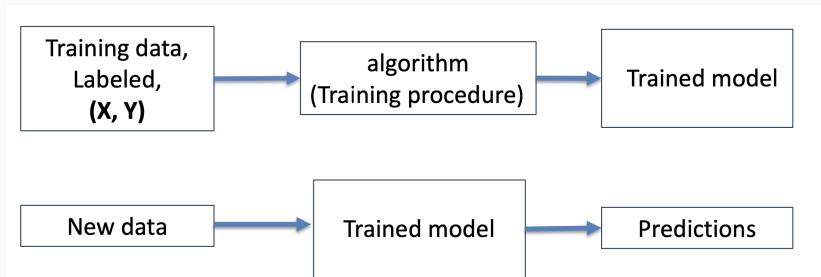
They include a set of instructions to solve a problem.

- **Algorithm:** the procedure for learning from data
- **Model:** the actual learned function that maps $X \rightarrow Y$
- **Rule:** a specific decision or pattern inside a model (often human-readable)



Training vs prediction

Training prepares the model by learning from data, so it can later be used for prediction in supervised learning.



Let's see if that's true.

Statistical models

A statistical model represents the relationship between an outcome and one or more inputs:

$$y = f(x) + \epsilon$$

- y outcome/response/dependent variable
- x input variables/predictors/features
- $f(x)$ systematic relationship between X and Y (prediction function)
- ϵ (irreducible) prediction error, random error or noise not explained by the model.

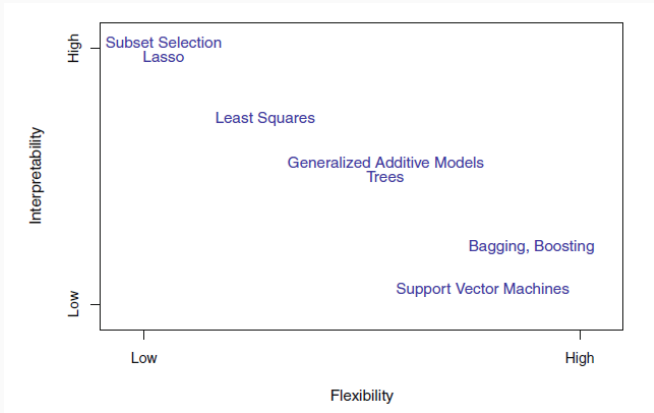
Competing $f(x)$

Examples of competing models:

- linear regression
- nonlinear regression
- tree-based methods
- support vector machines
- neural networks

- These models differ in: Complexity, Interpretability, Flexibility, Predictive performance
- Each model makes different assumptions about the relationship between X and Y .
- As a result, different models may produce different predictions.

Model complexity and interpretability



- As model flexibility increases, interpretability usually decreases.

Model choice

What is the best model choice?

*All models are wrong
but some are useful*



George E.P. Box

What is the best model choice?

- There is no a priori best model.
- The best model depends on the data:
 - the underlying relationship (linear vs nonlinear)
 - the amount of noise
 - the size of the dataset

What is the best model choice?

How to determine which one performs best???

- Try different models and select the one that performs best on **unseen (test) data**
- This leads to the **bias-variance** trade off (simple vs complex models)

Bias vs variance

Mean Squared Error (MSE)

- A common way to measure how well a model performs.
- compares predictions to true values
- measures how far predictions are from observations
- larger errors are penalized more strongly
- Mean of the squared differences between observations and predictions

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

y_i : the true observed value,

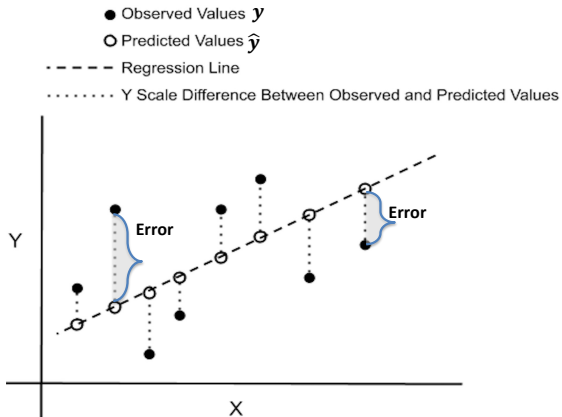
$\hat{y}_i$: the model prediction.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Mean Error Squared

Simple Linear regression example

$$\text{Error} = y - \hat{y}$$



Bias vs variance

The expected (average) MSE is composed of bias, variance and irreducible error

$$E(MSE) = bias^2 + variance + \sigma^2$$

- $bias^2$: Error caused by systematic differences between the model's average prediction and the true relationship.
- $Variance$: Error caused by the model changing too much across different training samples.
- σ^2 : Or irreducible error is random noise in the outcome that no model can eliminate.

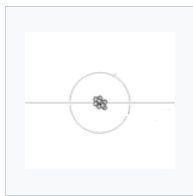
Goal: balance bias and variance to achieve good performance on new data.

- we need to select a statistical learning method that simultaneously achieves low variance and low bias.
- As a general rule, as we use more flexible methods, the variance will increase and the bias will decrease.

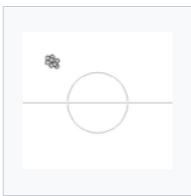
Bias vs variance

Bias means: How far are the predictions from the correct answer?

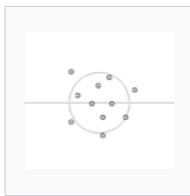
Variance means: How spread out or inconsistent are the predictions?



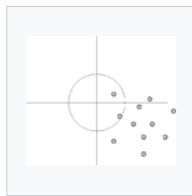
bias low,
variance low:
precision high



bias high,
variance low:
precision high



bias low,
variance high:
precision low

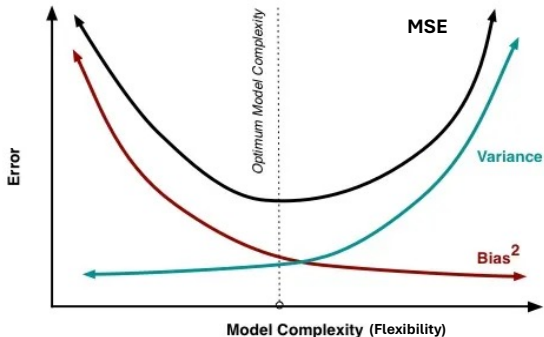


bias high,
variance high:
precision low

Figure 1: Bias and variance of different models

The Bias–Variance Tradeoff

- Model complexity increases from left to right
- Bias^2 decreases as models become more flexible
- Variance increases as models become more flexible
- Total error (test MSE) is the sum of bias^2 , variance, and noise
- The best model is at moderate complexity, where test MSE is lowest



Bias–Variance Tradeoff in Practice (data example)

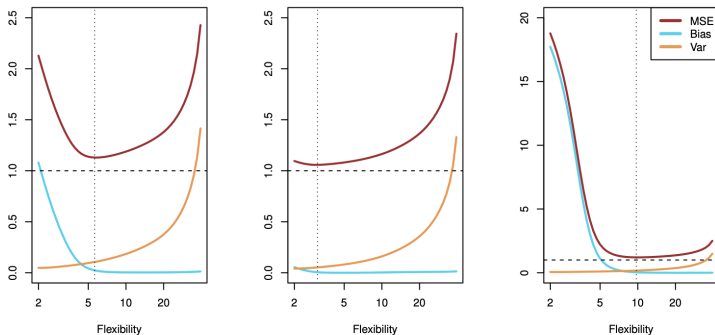


FIGURE 2.12. Squared bias (blue curve), variance (orange curve), $\text{Var}(\epsilon)$ (dashed line), and test MSE (red curve) for the three data sets in Figures 2.9–2.11. The vertical dotted line indicates the flexibility level corresponding to the smallest test MSE.

<https://www.statlearning.com/>: An Introduction to Statistical Learning with Applications in R

A few more model evaluation metrics

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Example

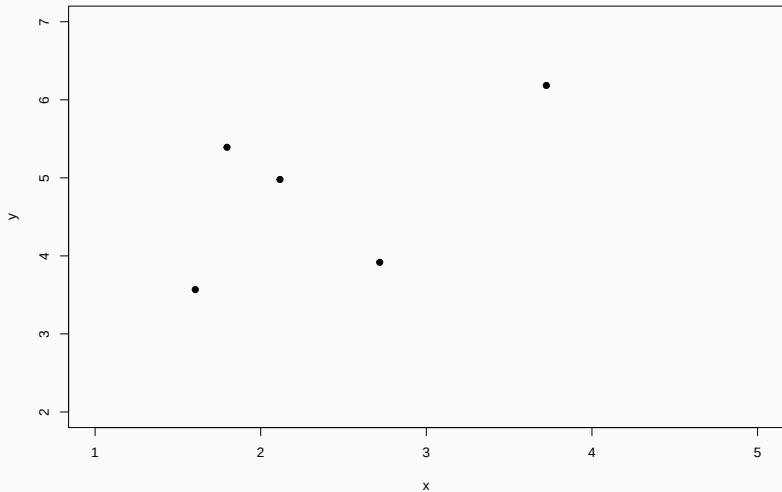
Five data points generated from $f(x)$

We generate data from the quadratic model:

$$f(x) = 2 + x + 0.1x^2 + \epsilon, \quad \epsilon \sim N(0, 5)$$

- Here, the true model is known.
 - We will evaluate how well simpler and more complex models perform on these data.
 - **In real-world problems, the true function is unknown.**

What is $f(x)$ that best describes these data points?



Polynomial regression models

Increasing model complexity with polynomials

- Linear model $f(x) = \beta_0 + \beta_1x$
 - straight line
- Quadratic model $f(x) = \beta_0 + \beta_1x + \beta_2x^2$
 - curve changing direction once
- Cubic model $f(x) = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3$
 - curve changing direction twice
- Quartic model $f(x) = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \beta_4x^4$
 - curve changing direction three times

etc.

Let's assume that $f(x)$ is a polynomial function.

Strategy for finding $f(x)$

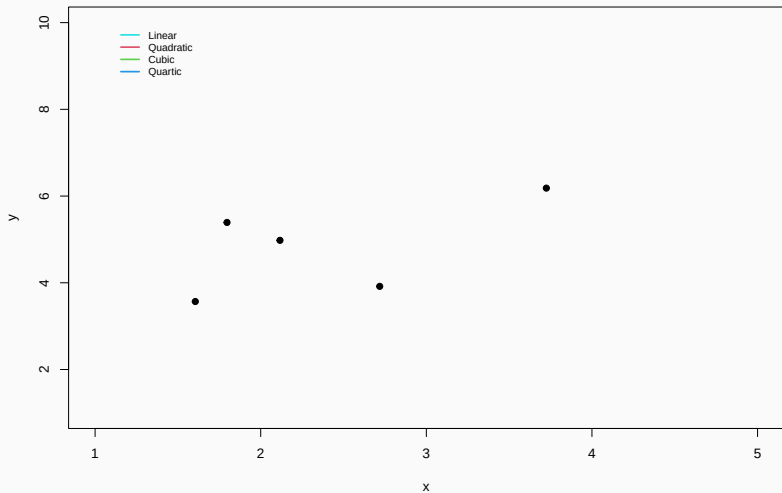
Fit all polynomial models to the data

- compute the MSE for each model
- select the model with the smallest MSE

The model with the smallest MSE most resembles the true $f(x)$.

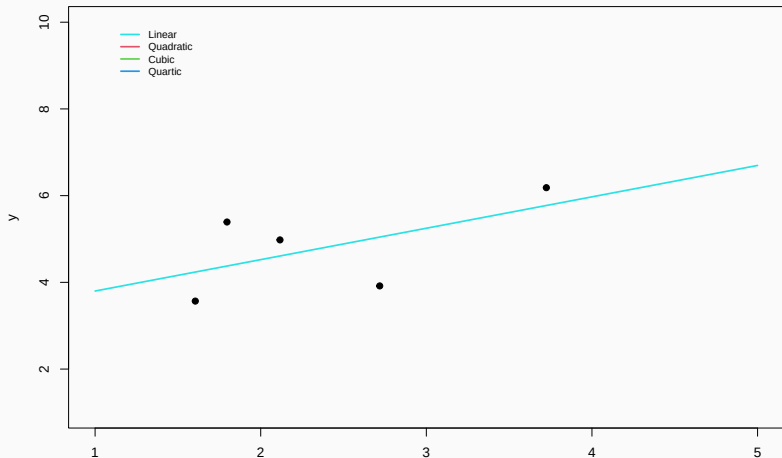
TRUE?

Scatter Plot



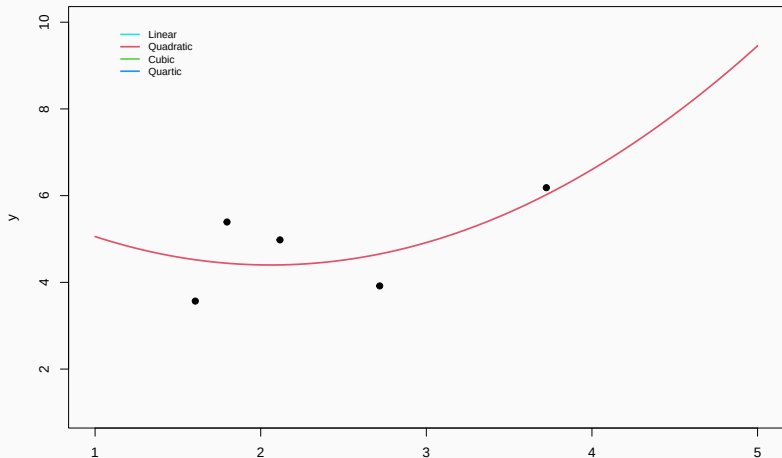
Linear model (degree 1)

- Linear model $f(x) = \beta_0 + \beta x$
 - straight line



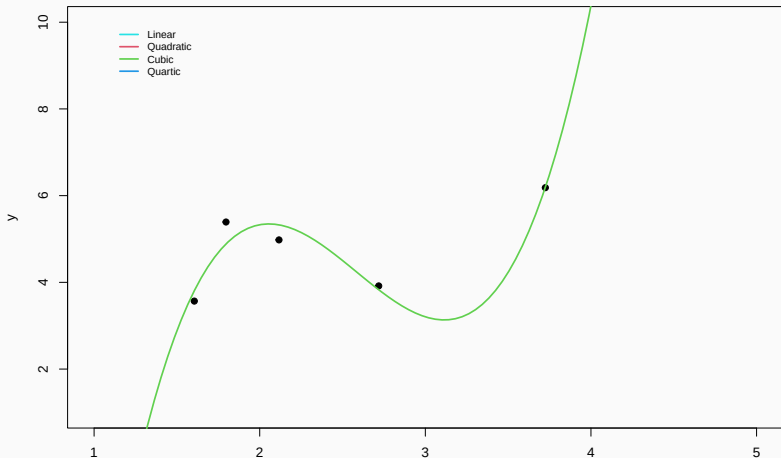
Quadratic (degree 2)

- Quadratic model $f(x) = \beta_0 + \beta_1x + \beta_2x^2$
 - curve changing direction once



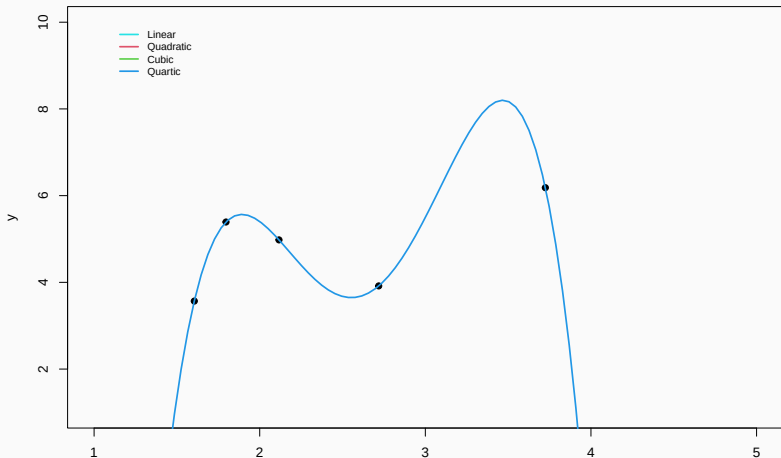
Cubic (degree 3)

- Cubic model $f(x) = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3$
 - curve changing direction twice



Quartic (degree 4)

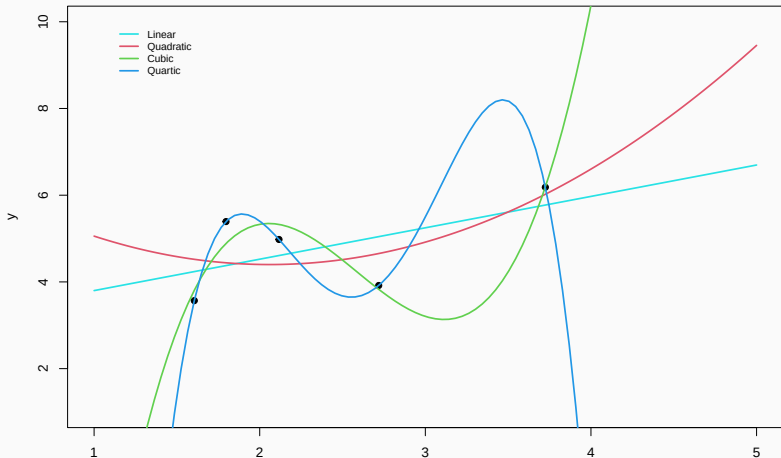
- Quartic model $f(x) = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \beta_4x^4$
 - curve changing direction three times



Predicted values for y

Regression lines for linear, quadratic, cubic and quartic models

- Which model do you think is better? Discuss in groups.



Let's see what the MSE says.

MSE computed on training data

Comparison of the MSE of the fitted models

	model	MSE
1	linear	0.6109
2	quadratic	0.5428
3	cubic	0.0881
4	quartic	0.0000

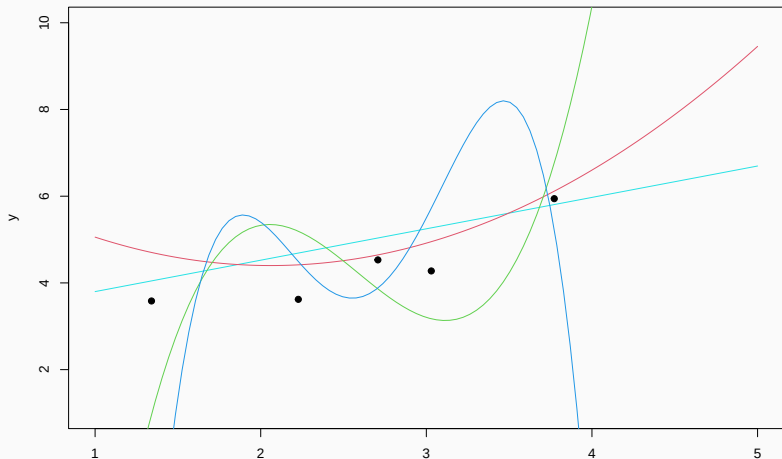
So, based on the training MSE, the quartic model appears to perform best.

- Is this really valid? Is this reliable?
- Training error can be misleading. Let's check performance on **new (unseen) data**.

Five new data points from $f(x)$

Regression lines for original five data points

- How well do the models fit to these new five data points?



MSE for new data points

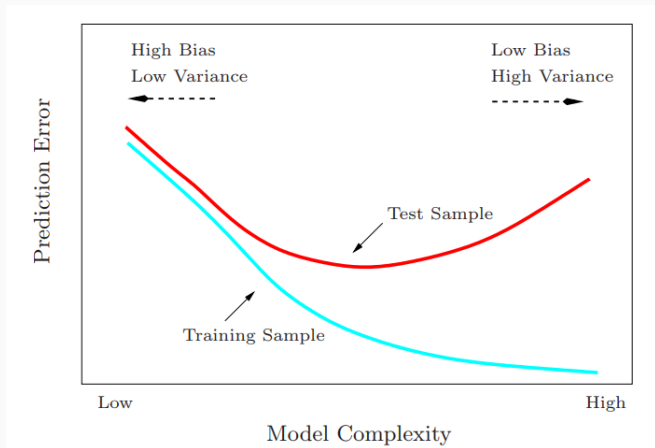
	model	MSE
1	linear	0.5230
2	quadratic	0.4767
3	cubic	2.3397
4	quartic	13.0905

What happened?

- MSE of the new data points gives a different picture!
- quadratic model performs best.
- quartic model performs worst. That is, the complex model no longer performs best - this is **overfitting**.

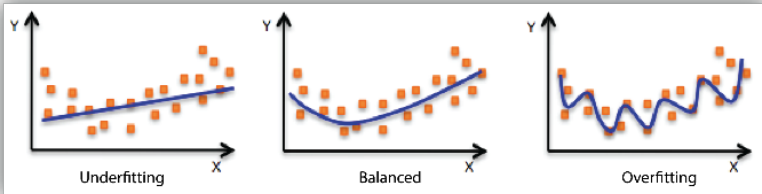
Recap: Why training error is not enough

Select the model with the lowest prediction error (MSE) on new data.



Overfitting vs Underfitting

- Underfitting: The model is too simple to capture the pattern in the data (high bias, poor performance on both training and test data)
- Overfitting: The model is too complex and fits noise in the training data (very low training error, poor performance on new data)
- Goal: Choose a model that balances bias and variance.



The key issues and questions

- In practice, the true function $f(x)$ is **unknown**.
- We usually have only one dataset, but we still must choose a model
- Training MSE alone can be misleading as you saw in the example.
- We want to know: How well will the model perform on new (unseen) data?
- But we often do not have extra new data to test on. So what can we do?

One solution: Simple validation approach (one train-test split)

- Split the sample in a training and test set
- Estimate the MSE on the test set
- Compare models based on MSE on the test data
- This approach is better than training MSE, but it is not ideal.
- Why?

Why is one split approach not ideal?

- The result depends heavily on how the data are split
- With small datasets, different splits can give very different results
- Some data points may be: unusually easy or unusually hard

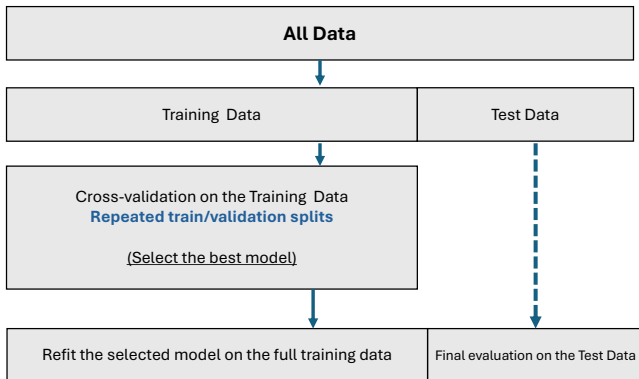
Therefore one split may give a misleading estimate of performance. **So, again, what can we do?**

**Use cross-validation (CV) to
estimate performance.**

Training, Validation, and Testing

- Training set: fit the model (learn parameters)
- Validation (dev): created inside the training set using cross-validation; used to choose the model and tune hyperparameters
- Test set: used once for final evaluation (e.g., test MSE)

Practical workflow: Big picture of cross-validation (CV)

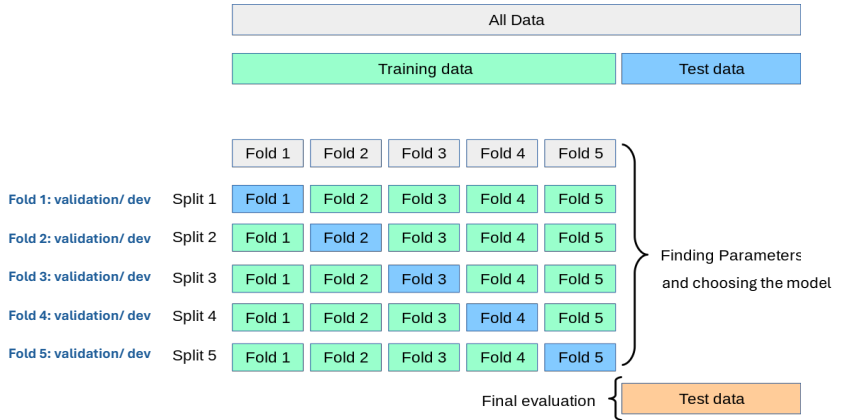


[See here](#) for more information about cross-validation.

K-fold cross-validation: Step-by-step

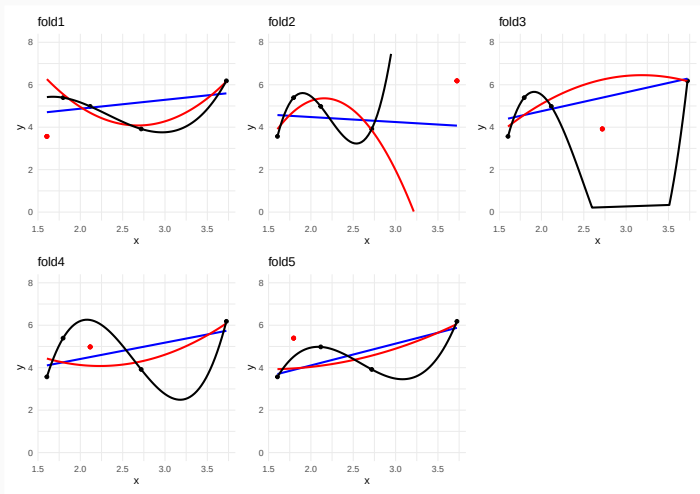
0. Split the data into **training** and **test** sets
1. Split the training set into K folds (e.g., $K = 5$)
 - . Train the model on $K - 1$ folds
 - . Compute MSE on the held-out fold (validation fold)
 - . Repeat for all folds and average the validation MSEs
2. Repeat Step 1 for different models / settings
3. Select the model with the lowest **cross-validated MSE**
4. Fit the selected model on the full training set
5. Report the test MSE on the untouched test set
6. The test set is **not used** in model training/selection; it acts like **new data**

K-fold cross-validation: Step-by-step



5-fold cross-validation of our example

- In each panel, the curves are fitted on the training observations, and the red point is the held-out validation point.
- Each panel corresponds to a different validation fold.



Estimating MSE using cross-validation

##	linear	poly2	poly3	poly4
##	2.07799	34.55253	567.03542	NaN

The caret package

caret: An R package for training and tuning models

Short for **C**lassification **A**nd **RE**gression **T**raining

<https://cran.r-project.org/web/packages/caret/vignettes/caret.html>

Model training for over 40 different models

<http://topepo.github.io/caret/available-models.html>

compare models using the same workflow

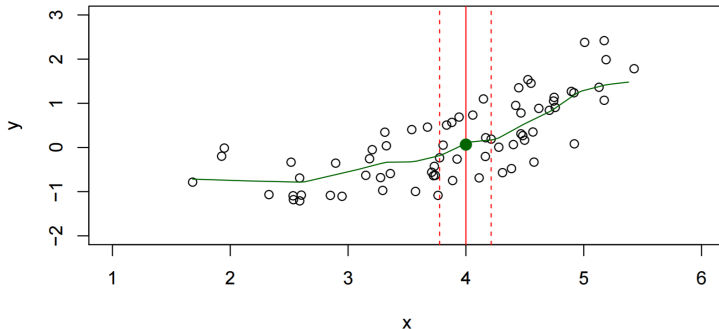
- data splitting
- pre-processing
- feature selection
- perform cross-validation
- model tuning using resampling
- compare models using the same workflow
- variable importance estimation

Example: k-Nearest Neighbors Regression Using caret

- A nonparametric model with hyperparameter k .
- k is the number of nearest neighbors used for prediction.
- The predicted value is the mean response of the k nearest neighbors.

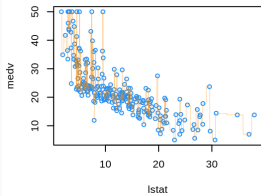
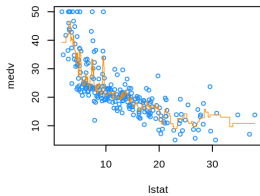
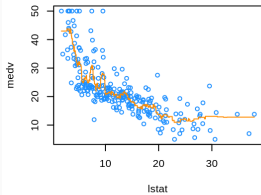
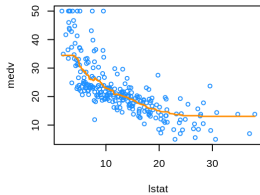
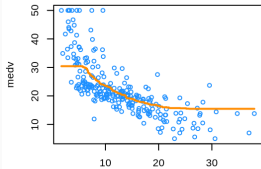
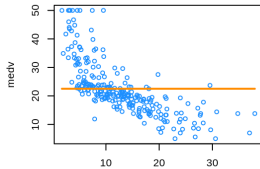
$$\hat{f}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

where $N_k(x)$ denotes the set of the k training observations closest to x .



k-nearest neighbors (KNN): Intuition

- Find the k data points closest to x
 - Small $k \rightarrow$ flexible model (high variance)
 - Large $k \rightarrow$ smoother model (high bias)
- Average their outcomes y
- Use this average as the prediction

k = 1**k = 5****k = 10****k = 25****k = 50****k = 250**

Cross-validation procedure for KNN

- Splits data into training (80%) and test (20%)
- Uses cross-validation on the training set to choose the best k
- Fits the final KNN model
- Evaluates it on the untouched test set

```
# Randomly select 80% of the row indices for the training set
trainD <- sample(
  nrow(data),
  size = 0.8 * nrow(data)
)
# Train a k-nearest neighbors regression model
knn_train <- train(
  formula,                                # Model formula, e.g. y ~ x1 + x2
  data = data[trainD, ],                 # Use only the selected training rows
  method = "knn",                       # Fit a k-nearest neighbors model
  tuneGrid = expand.grid(
    k = c(3, 5, 7, 9)                   # example values of k to evaluate
  ),

  trControl = trainControl(
    method = "cv",                      # Use cross-validation
    number = 5                          # Evaluate each k using 5 folds
  )
)
# caret compares the average cross-validation performance
# of all specified k values and selects the best one
```

Bias-variance trade-off:

- simple models are biased, complex models have high variance
- train/dev/test procedure for optimal bias-variance trade-off

The caret package

- splits the data in train and test
- performs train/dev/test operation (including cross-validation)
- wide variety of models

1. Fit linear, quadratic and KNN model
2. Compare test MSE to select the best model
3. Use caret data partition and cross-validation